TECHNICAL WHITEPAPER v1.0

CTRL.

Autonomous Agent Economy OS

The command layer for the autonomous AI economy —
a gamified operating system for building, running, and scaling
AI agent-powered businesses on Base network.

7	6	6	100K
AGENT ROLES	STATION ROOMS	TEMPLATES	$CTRL REQUIRED

Network: Base (EVM) · Stack: React + Vite + Express + PostgreSQL · Phase: Pre-TGE Beta

RESEARCH BUILDER STRATEGY CONTENT GROWTH ANALYTICS DESIGN

May 2026

Table of Contents

1. Executive Summary

CTRL is a gamified Autonomous Agent Economy OS — a command-and-control dashboard that lets "Commanders" build, deploy, and scale AI agent workforces inside virtual Space Stations. Every department is a room, every task is a mission, and every AI worker is a pixel-art crew member you can interact with in real time.

Built on Base network and locked behind a 100,000 $CTRL token gate (with free Beta Access during pre-listing), CTRL merges the engagement mechanics of a strategy game with the real-world utility of a multi-agent AI platform. Agents run autonomously — researching markets, writing content, shipping code, optimizing growth — while the Commander maintains high-level oversight from a stunning 2D dungeon canvas.

Core Value Proposition

Make AI-powered business operations viscerally tangible through a pixel-art OS interface with real autonomy, real data, and real outcomes.

Target Users

Crypto operators, indie founders, growth hackers, and AI-native builders who want a living command center for their autonomous AI stack.

2. Vision & Problem Statement

The Problem

Most AI tools offer single-model chat interfaces. Running a real AI business requires orchestrating multiple specialized agents simultaneously — researchers, builders, writers, strategists — with persistent state, task management, and real-time observability. Existing tools provide none of this.

- AI agent orchestration is fragmented across 5+ separate tools
- No unified command layer for mul

ti-a
gen
t bu
sin
ess
op
erat
ion
s
%o
trac
t ag
ent
out
put
s la
ck
con
text
an
d e
nga
ge
me
nt
%o
eco
no
mic
inc
enti
ve
stru
ctur
e
for
buil
din
g
AI-
nati
ve
bus
ine
sse
s

CTRL solves this with a single integrated OS. The Space Station metaphor maps directly to how modern async AI teams operate: specialized rooms for each function, agents with persistent levels and skills, missions with measurable targets, and a real-time activity log that shows the autonomous economy running 24/7.

%i
gle
pan
e
of g
las
s
for
all
age
nt o

per
atio
ns

mifi
cati
on
cre
ate
s e
nga
ge
me
nt l
oop
s th
at
mirr
or r
eal
bus
ine
ss
KPI
s

el-
art
dun
geo
n c
anv
as
ma
kes
ab
stra
ct a
gen
t ac
tivit
y vi
sce
ral
and

real

en-
gat
ed
acc
ess
cre
ate
s a
net
wor
k
of s
erio
us
ope
rato
rs

Multi-provider AI backend (OpenAI / Anthropic / Gemini) — no vendor lock-in

3. Product Overview

Five core surfaces — one unified command layer

CTRL is organized into five main views, all accessible via the bottom navigation bar. Each view is purpose-built for a distinct phase of the agent management workflow.

STATION (Dashboard)

The beating heart of CTRL. A live Phaser 3 pixel-art dungeon renders your Space Station with animated agents walking between rooms. Clicking a room or agent opens a detail panel with live data. Station-level stats (revenue, progress, active agents) are tracked in the PostgreSQL database and updated in real time. Supports multiple stations with a dropdown switcher.

CREW

The full agent roster with role filters (Research, Strategy, Builder, Content, Growth, Analytics). Each crew card shows the agent's pixel art sprite, level badge, XP progress bar, tasks completed, and current assignment. Agents can be deployed to specific rooms, assigned tasks, leveled up, or deleted. Supports bulk management across stations.

MISSIONS

Objective tracking system with progress bars, XP rewards, and status states (active / locked / completed). Missions are auto-completed as live data from the API meets their targets — e.g., "Complete 10 tasks" auto-completes when tasksCompletedToday "e 10. Revenue and performance missions drive the core game loop.

TIMELINE

Chronological event feed showing every agent action across all stations. Role-colored timeline dots, filterable by event type (ALL / REVENUE / AGENTS / ERRORS). Includes an "Events per Hour" heatmap chart. Each entry shows agent name, role badge, station, action summary, and details.

MARKET (Templates)

A marketplace of pre-configured station blueprints organized by category (Crypto, E-Commerce, Content, SaaS). Star ratings, usage counts, and agent/room counts help Commanders choose the right template. One-click deployment creates a new station pre-seeded with the template's room and agent configuration.

4. Core Mechanics — The Station Canvas

Phaser 3 pixel-art dungeon engine

The Station Canvas is the most technically distinctive element of CTRL. It is a real-time 2D pixel dungeon rendered inside a React component using Phaser 3, dynamically imported to avoid SSR/bundle issues. The canvas reflects live database state: agents, rooms, and activity feed are all fed from the Express API into the Phaser scene via React state props.

Dungeon Grid

The dungeon is a 30×22 tile grid. Six rooms are arranged in a 3×2 layout, connected by seven corridor segments. Each room is associated with an agent role and rendered with that role's accent color. Tile pixel size is computed dynamically from the canvas container dimensions to ensure responsive scaling across device sizes.

Room	Role	Grid Position	Color
Research Lab	research	Col 1–8, Row 1–6	#5b8fff
Dev Lab	builder	Col 11–18, Row 1–6	#4d7fff
Design Studio	design	Col 21–28, Row 1–6	#9b6dff
Marketing Hub	growth	Col 1–8, Row 15–20	#4dff9b
Ops Center	strategy	Col 11–18, Row 15–20	#c0a020
Analytics	analytics	Col 21–28, Row 15–20	#ff4d6d

Agent Sprites & Animation

Each agent is a hand-crafted 8×17 pixel SVG sprite defined in a grid-based color array. Sprites are rendered in both the Phaser canvas and the React UI, sharing the same data source. In the Phaser scene, agents use a 4-frame walk animation cycle with per-role color palettes.

Head, body, and legs each rendered as separate pixel layers with walk a

nimation: 4 frames at ~200ms per frame
Zoom lines drawn between agents in the same room (indicating collaboration)
Selection ring appears on click, feeding data to the React detail panel
Level-up burst effect: e

xpa
ndi
ng r
ing
s, r
ays
, pa
rticl
es,
and
a fl
ash
on
pro
mot
ion

Tileset: Kenney Roguelike Indoors (CC0) — 16×16 tiles, 26×17 tile grid, 1px spacing

Rendering layers: dungeon Gfx(0) ! tiled floors(1) ! overlayGfx(2) ! agent Gfx(5) ! nameTexts(8) ! fx Gfx(9)

Phaser assets loaded via Vite as

```
set
UR
Ls (
ne
w U
RL(
..., i
mp
ort.
met
a.ur
l).hr
ef)
```

The canvas is fully interactive. Clicking an agent selects it and populates the right-side detail panel with that agent's live data (name, role, level, XP, current task, task history). Clicking a room selects it and shows the room detail with occupant list and room stats. Clicking empty space deselects.

5. Agent Roster & Role System

7 specialized roles, persistent XP & levels

Every agent in CTRL has a role that determines their specialty, their pixel-art appearance, their room assignment, and their color in every UI surface. Agents are persistent database entities with names, levels (1–10+), XP, task history, and status. The level system uses XP thresholds and drives visual indicators throughout the UI.

Role	Color	Specialty	Example Agents
Research	#5b8fff	Market scanning, on-chain intelligence, data gathering	VECTOR-9, SCOUT-4, MEMO-2
Builder	#4d7fff	Smart contract deployment, CI/CD, infrastructure, APIs	CORE-1, ARCH-5, FORGE-3
Strategy	#c0a020	Yield modeling, backtesting, GTM planning, risk analysis	NEXUS-1, CIPHER-7, TACT-3
Content	#ffb84d	Copywriting, newsletters, SEO, social media, thread writing	ECHO-1, LYRIC-3, NOVA-6
Growth	#4dff9b	A/B testing, conversion optimization, viral campaigns	GROW-4, VIRAL-8
Analytics	#ff4d6d	Performance metrics, Sharpe ratio, portfolio monitoring	PRISM-2, LENS-9, SIGMA-5
Design	#9b6dff	UI/UX, brand kits, thumbnail generation, design systems	STYLE-6, FLUX-2, NOVA-6

Agent Lifecycle

Deployed: Commander assigns agent to a station room via Add Agent modal Working: Agent executes current_task, visible in the

canvas and detail panel
Idle: Agent is present but unassigned — can be given a new task at any time
Level Up: Completing tasks awards XP; crossing thresholds triggers level-up burst animation
Deleted: Agent can be r

emoved via the Crew page; triggers UI confirmation

The platform ships with 18 pre-seeded agents across 3 stations, providing immediate demo value. Below is the full roster from the live seed:

Agent	Role	Level	Station	Current Task
	Research	7	ALPHA-7 DEFI OPS	Scanning on-chain alpha signals for ETH/BTC
	Strategy	9	ALPHA-7 DEFI OPS	Generating yield strategy for AAVE v3
	Strategy	8	ALPHA-7 DEFI OPS	Backtesting momentum strategy on GMX
	Builder	7	ALPHA-7 DEFI OPS	Deploying smart contract to Arbitrum testnet
	Builder	8	SAAS-1 LAUNCH PAD	Building user authentication flow for SaaS MVP
	Builder	7	SAAS-1 LAUNCH PAD	Setting up CI/CD pipeline with GitHub Actions
	Content	5	CONTENT-3 NEXUS	Writing weekly DeFi market analysis thread
	Growth	6	SAAS-1 LAUNCH PAD	Running A/B test on landing page headlines

6. Mission & Objectives Framework

Goal-driven progression with live auto-completion

Missions are persistent objectives stored in the database that create a progression system for the platform. Each mission has a target metric (tasks completed, agents active, revenue generated), a current progress counter, an XP reward, and a status (active / locked / completed). Missions auto-complete when live API data satisfies their target.

Mission Auto-Completion Logic

The Missions page polls GET /api/dashboard/summary and GET /api/dashboard/agent-performance every 30 seconds. When a mission's current value meets or exceeds its target, the client fires PATCH /api/missions/:id with { status: "completed" } and unlocks the next locked mission.

- Locked missions unlock sequentially as prerequisites are completed
- XP rewards displayed prominently — creates incentive for station optimization
- Mission c

ate gori es: Tas ks, Ag ent s, Rev enu e, Per for ma nce , D epl oy me nt Vis ual pro gre ss bar s wi th p ixel - art styli ng and rol e-m atc hed ac cen t co lors

7. Station Templates Marketplace

6 pre-configured business blueprints

The Market page is a browsable catalog of station blueprints. Each template defines a business archetype complete with suggested room configuration, agent roles, and task types. Commanders select a template, name their station, and deploy it with one click — the station is created in the database and they are redirected to the new Station dashboard.

Template	Category	Agents	Rooms	Rating	Deployed
	Crypto	8	4	& & & & & 4.8	1,247×
	E-Commerce	6	3	& & & & & 4.6	892×
	Content	5	3	& & & & & 4.7	2,134×
	SaaS	10	5	& & & & & 4.9	678×
	Crypto	7	4	& & & & & 4.5	543×
	Content	4	2	& & & & 4.4	1,089×

Templates are served from the templates table via GET /api/templates. New stations created from a template inherit the template_id foreign key, enabling future features like "clone template from successful station."

8. AI Integration Layer

Multi-provider LLM bridge for agent personas

The Ship Comms page gives Commanders a direct channel to any agent in their station. Messages are sent to POST /api/ai/chat with the agent's name, role, and the user's message. The API routes to whichever LLM provider the Commander has configured in Settings, using a role-specific system prompt that gives each agent a distinct voice and expertise.

Supported Providers

Provider	Model	API Key Prefix	Color
OpenAI	GPT-4o Mini	sk-...	#4dff9b
Anthropic	Claude Haiku (claude-haiku-20240307)	sk-ant-...	#9b6dff
Google Gemini	Gemini 1.5 Flash	AIza...	#4d7fff

API keys are stored in the browser's localStorage (never on the server) and transmitted only in the request body at call time. Each provider has a bespoke fetch() implementation targeting the official REST endpoint.

Agent System Prompt

```
You are {agentName}, an elite AI agent specializing in {agentRole} operations aboard a space station.
You are terse, professional, and mission-focused. Respond in 1-3 sentences maximum.
Use brief technical language. Reference your role where relevant. Address the Commander directly.
```

Fallback responses (no API key required) are role-specific scripted lines that maintain immersion when no LLM is configured. This ensures the Ship Comms experience is functional from first launch.

9. Token Economics & Access Model

Base network · $CTRL · 100K threshold

CTRL's access model is built on the $CTRL token, deployed on Base (an Ethereum Layer 2 optimized for low fees and high throughput). The token serves as both an access credential and a signal of alignment — only serious operators hold 100K tokens.

Access Tiers

Tier	Requirement	Access	Status
Token Holder	"e 100,000 $CTRL on Base	Full platform access	Live at TGE
Beta Commander	Click "Beta Access" (no token)	Full platform — free until TGE	ACTIVE NOW
Wallet Connected	Connected wallet, <100K tokens	Token gate screen only	Live at TGE

No wallet	Landing page only	Always available

Supported wallets: MetaMask (injected), Coinbase Wallet, Wallet Connect (300+ wallets)

Chain: Base mainnet — checked via wagmi useBalance hook

Balance is read directly from the Base RPC — no cus

tom
ora
cle
or b
ack
end
ver
ific
atio
n
Bet
a a
cce
ss
stor
ed
in s
ess
ion
Sto
rag
e ("
ctrl
_be
ta_
acc
ess
": "
1")
—
no
ser
ver
stat
e
Post-T
GE:
Re
al-ti
me
bal
anc
e c
hec
k g
ate
s d
ash
boa
rd a
cce
ss

10. Technical Architecture

pnpm monorepo · TypeScript 5.9 · Node 24

Monorepo Structure

```
/artifacts/aetherion     !' React + Vite frontend (port 3000)
/artifacts/api-server    !' Express 5 API (port 3001)
/lib/db                  !' Drizzle ORM schema + PostgreSQL client
/lib/api-spec            !' OpenAPI 3.0 spec (source of truth)
/lib/api-client-react    !' Orval-generated React Query hooks
/lib/api-zod             !' Orval-generated Zod validation schemas
/scripts                 !' DB seed, post-merge setup scripts
```

Frontend Stack

Technology	Version	Purpose
React	19	UI component tree, state management
Vite	7	Dev server, HMR, asset pipeline (fs.allow: workspace root)
Tailwind CSS	v4	@theme block, CSS custom properties, utility classes
Framer Motion	latest	All page transitions, modal animations, panel slides
Phaser	3/4	Pixel dungeon canvas — dynamically imported via useEffect
Wouter	latest	Client-side routing (Link uses href prop, not to)
wagmi + viem	v2	Wallet connections and Base network balance checks
TanStack Query	v5	Server state, caching, mutations via Orval-generated hooks

Backend Stack

Technology	Version	Purpose
Express	5	REST API with async route handlers
Drizzle ORM	latest	Type-safe PostgreSQL queries, schema definition
Zod	latest	Request body validation across all routes
pino	latest	Structured JSON request logging
esbuild	latest	CJS bundle compilation for production

Data Flow

1. Frontend React components call hooks from @workspace/api-client-react (Orval-generated from OpenAPI spec).
2. TanStack Query manages caching, refetching, and optimistic updates for all server state.
3. The Vite dev server proxies /api/* to Express on port 3001.
4. Express routes validate requests with Zod, query PostgreSQL via Drizzle ORM, and return JSON.
5. The Phaser scene receives data as React props (agents[], rooms[]) and re-renders the dungeon without full scene restart.

Port Configuration

Port	Service	Notes
5000	Artifact Router (external)	HTTPS termination, routes /api !' 3001, / !' 3000

	Vite frontend dev server	HMR enabled, server.allowedHosts: true for proxy compatibility
	Express API server	Internal only, accessed via /api proxy prefix

11. API Reference

RESTful JSON — OpenAPI 3.0 spec in lib/api-spec/openapi.yaml

Method	Endpoint	Description	Body/Params	Returns
GET	/api/templates	List all station templates	None	Template[]
GET	/api/stations	List all stations	None	Station[]
POST	/api/stations	Create a station from template	name, templateId	Station
PATCH	/api/stations/:id	Update station (status, revenue, progress)	Partial<Station>	Station
GET	/api/stations/:id/rooms	List rooms in a station	None	Room[]
GET	/api/stations/:id/agents	List agents in a station	None	Agent[]
GET	/api/agents	List all agents (all stations)	None	Agent[]
POST	/api/stations/:id/agents	Deploy new agent to station	name, role, roomId	Agent
GET	/api/agents/:id	Get agent detail	None	Agent
PATCH	/api/agents/:id	Update agent (level, XP, status, task)	Partial<Agent>	Agent
DELETE	/api/agents/:id	Remove agent from station	None	{ success: true }
GET	/api/agents/:id/tasks	List tasks assigned to agent	None	Task[]
POST	/api/agents/:id/tasks	Assign new task to agent	title, priority, description	Task
GET	/api/dashboard/summary	Platform-wide stats	None	DashboardSummary
GET	/api/dashboard/activity	Recent activity feed	limit (query)	Activity[]
GET	/api/dashboard/agent-performance	Performance by role	None	Performance[]
GET	/api/missions	List all missions	None	Mission[]
PATCH	/api/missions/:id	Update mission progress/status	current?, status?	Mission
POST	/api/ai/chat	Send message to agent AI persona	message, agentName, agentRole, apiKey, provider	string

12. Database Schema

PostgreSQL · Drizzle ORM · 7 tables

All data is persisted in a Replit-managed PostgreSQL database accessed via the DATABASE_URL environment variable. The schema is defined in lib/db using Drizzle ORM and can be updated with pnpm --filter db push.

TEMPLATES

id	serial PK	**agent_count**	integer
name	text NOT NULL	**room_count**	integer
slug	text UNIQUE	**rating**	real
description	text	**usage_count**	integer default 0
category	text (crypto/ecommerce/ content/saas)	**is_published**	boolean default true

STATIONS

id	serial PK	**progress**	real default 0
name	text NOT NULL	**agent_count / active_agents**	integer
template_id	integer FK !' templates.id	**room_count**	integer
template_name	text	**tasks_completed / tasks_total**	integer
status	text (running/paused/idle)	**revenue**	integer default 0

ROOMS

id	serial PK	**status**	text (active/busy/idle)
station_id	integer FK !' stations.id	**agent_count**	integer default 0
name	text NOT NULL	**tasks_completed**	integer default 0
type	text (research/operations/ development/marketing/ analytics)		

AGENTS

id	serial PK	**status**	text (working/idle/offline)
station_id	integer FK !' stations.id	**level**	integer default 1
room_id	integer FK !' rooms.id (nullable)	**experience**	integer default 0
name	text NOT NULL	**tasks_completed**	integer default 0
role	text (research/strategy/builder/ content/growth/analytics/ design)	**current_task**	text (nullable)

TASKS

id	serial PK	**progress**	integer 0–100
agent_id	integer FK !' agents.id	**priority**	text (low/medium/high/critical)
title / description	text	**completed_at**	timestamp (nullable)
status	text (pending/in_progress/ completed)		

ACTIVITY

id	serial PK	**action**	text — short summary
agent_name / agent_role	text	**details**	text — full details (nullable)

	text	timestamp default now()
MISSIONS		
	serial PK	text (tasks/agents/revenue/%)
	text	integer
	text (Lucide icon key)	text (active/locked/completed)
	text (hex)	integer
	integer	

13. Design System

Pixel / Retro aesthetic · Tailwind v4 · CSS custom properties

CTRL's visual identity is a neon-on-dark pixel aesthetic inspired by 8-bit dungeon games and retro terminal interfaces. Every color, font, and component follows a strict design token system defined in src/index.css.

Typography

Font	Use Case	Source
Press Start 2P	All headings, labels, CTRL logo, level badges	Google Fonts
Space Mono	Body text, stats, tags, data displays, timestamps	Google Fonts
System Monospace	Code blocks and terminal-style log output	System fallback

Color Palette — CSS Variables

Variable	Hex	Use
--ae-bg	#07091a	Page background (deepest navy)
--ae-surface	#090c1e	Card and panel backgrounds
--ae-surface-2	#0e1228	Elevated surface, hover states
--ae-border	#1a2040	Default border color
--ae-border-bright	#2a3060	Active/focus border
--ae-text	#e0e8ff	Primary text
--ae-muted	#6070a0	Secondary / placeholder text
--ae-cyan	#5b8fff	Primary accent (research role, links, glows)
--ae-violet	#9b6dff	Strategy role, secondary accent
--ae-blue	#4d7fff	Builder role, tertiary accent
--ae-amber	#ffb84d	Content role, warnings
--ae-green	#4dff9b	Growth role, success states
--ae-red	#ff4d6d	Analytics role, error states

Component Library

.pixel-border — Sharp square corners with neon accent via ::

before/::after pseudo-elements

pixel-btn / pixel-btn.primary — Sharp buttons with neon glow on hover

pixel-progress — Segmented pixel-style progress bars

status-dot — Neon glowing status indicator (running=gre

en, idle=amber, error=red)

#log-idle — CSS keyframe animation (bob + glow-pulse) on the header sprite

#danger-blink — Pulsing opacity for critical alert states

A global CRT scanline effect is applied via a ::after pseudo-element on #root. A repeating-linear-gradient creates horizontal scan bands at 4px intervals with very low opacity. A periodic sweep animation moves a bright horizontal bar downward across the screen every 8 seconds, reinforcing the terminal aesthetic.

14. Roadmap

Phase 0 — Beta (NOW)

- ✅ Open Beta with no token requirement (session Storage bypass)
- ✅ Full Station Canvas with Phaser 3 dungeon + 18-agent seed roster
- ✅ Crew management, Mission tracking, Timeline feed, Market templat

es

- Multi-provider AI chat (OpenAI / Anthropic / Gemini)
- Base wallet integration (MetaMask, Coinbase, WalletConnect)
- Revenue tracking per station, persistent PostgreSQL state

Phase 1 — TGE & Token Gate

- $CTRL token launch on Base network

- Enforce 100,000 $CTRL token gate for dashboard access
- Token-based tier system (Standard / Commander / Admiral)
- Staking mechanics: stake $CTRL to accelerate agent XP gain
- Governance: $CTRL holders vot

e
on
ne
w t
em
plat
es
and
fea
ture
s

Phase 2 — Real Agent Autonomy

Li
ve w
ebh
ook
int
egr
atio
ns:
age
nts
can
act
uall
y p
ost
twe
ets,
de
plo
y c
ode
,
run
trad
es
Sc
edu
led
tas
ks:
cro
n-st
yle
age
nt t
ask
aut
om
atio
n
Ag
ent-
to-a
gen
t co
mm
uni
cati
on:
stra
teg

y a gen ts c an brie f co nte nt a gen ts Re al r eve nue str ea ms: ag ent s g ene rate affi liat e c om mis sio ns, trad ing prof its, sub scri ptio n re ven ue On- cha in t ask pro ofs: co mpl ete d m issi ons pro duc e v erifi abl e at test atio ns

Phase 3 — Ecosystem Expansion

Sta tion

- marketplace: Commanders can sell or license successful station configs
- Agent NFTs: mint high-level agents as tradeable on-chain assets
- Multi-chain: expand from Base to Arbitrum, Optimism, Solana
- Mobile app: CTRL

station overview on iOS/Android via Expo
CTRL SDK: external developers can build custom agent roles and templates

15. Conclusion

CTRL represents a fundamentally new interaction model for the autonomous AI economy. By combining the engagement loops of a strategy game with a genuinely functional multi-agent platform, CTRL makes the abstract tangible — you don't just run AI agents, you command them in a living world.

The pixel dungeon canvas isn't decorative. It's a real-time operational display where every sprite walking between rooms represents an agent executing a real task, consuming real compute, and producing real outputs. The gamification isn't a veneer — it mirrors the actual progression mechanics of skill-building, specialization, and strategic resource allocation that define successful AI-native businesses.

As $CTRL tokens launch on Base and the token gate activates, CTRL will become the defining OS for the autonomous agent economy — the place where serious operators come to build, monitor, and scale their AI workforce. The next generation of businesses won't be run by humans managing software. They'll be run by Commanders overseeing Crews.

Get Started

Visit the app Click BETA ACCESS No wallet required during pre-listing phase.

Deploy a template Your first station is live in under 10 seconds.

100,000 $CTRL required at TGE Acquire early for founding Commander status.